

AI CRITIQUE OF THE USE CASE MODEL

Lab 1 deliverable 5 (procedure §3.5). Version 0.1, 2026-09-02. **Export this file to PDF before submission** — §4 of the lab manual asks for a PDF report.

1. What was submitted to the AI

AI tool	Claude (Opus 5), via the Claude Code interface
Date	2026-09-02
Attachment 1	DengueGuard-SG-UseCases.png — the master use case diagram (9 actors, 38 use cases), rendered from use-case-diagram.puml with PlantUML 1.2024.7
Attachment 2	REQUIREMENTS.md v0.3 — 349 atomised requirements in Fox form, §§1–11
Prompt (verbatim, as specified in lab §3.5.1)	<i>"Critique the attached use case diagram image, with respect to how well it matches the attached requirements file."</i>

No follow-up prompts were used. The response in §2 is the first and only response.

2. The AI's response (verbatim)

Coverage is broad and the traceability is unusually explicit. All eleven requirement sections are represented somewhere in the diagram, the system boundary is drawn correctly with the four external services (NEA, Weather, Telegram, OneMap) outside it, and the actor generalisation of Resident and Cleaning Crew Member to Registered User correctly factors out sign-in and password reset. The two notes anchoring 4.1.17 and 8.5.3 to the diagram are good practice — most student models leave the causal loop implicit.

Five mismatches against the requirements file:

1. 6.7 Raise Issue on Job is modelled as extending 6.6 Complete Job with Evidence, but requirement 8.3.8 says an issue may be raised "at any time before completion." An <<extend>> relationship means the extending behaviour is inserted at an extension point *within* the base use case's flow. As drawn, a crew member can only raise an issue while completing the job — which is precisely the case the requirement excludes, since a crew member who finds an obstruction has by definition not completed anything. The base should be the job-execution use case (6.5 Accept and Start Job), or 6.7 should stand alone as an association from Cleaning Crew Member.

2. 4.3 Notify Resident is included only by 7.6 Evaluate Alert Triggers, but three other requirements oblige resident notification. 5.2.8 requires notification on *every* change of a report's status, 8.5.2 on report closure, and 6.1.x only covers location alerts. As drawn, 3.4 Moderate Report and 6.8 Verify Completed Work change report status without any notification path. Either those two use cases should <<include>> 4.3, or the diagram understates the notification requirement.

3. There is no notification path to the Cleaning Crew Member at all. Requirement 8.2.4 obliges the system to notify an assignee within one minute of assignment, 8.2.6 on reassignment, and 8.3.11 when a completion is rejected. The only notification use case in the model is named 4.3 Notify Resident. Three obligations have no representation.

4. Section 9 (Map, Trend and History) is modelled for the

Operations Manager but not for the Resident. 5.3 `Inspect Cluster Detail` is associated with Operations Manager only, yet 9.1.1, 9.1.2, 9.1.5, 9.1.7 and 9.1.9 describe map, tier colouring, saved-location display and the 30-day trend, none of which are restricted to managers by any requirement in §2.3. The resident-facing map — which §11.2's screen inventory does list — has no use case.

5.2.1.12 (log out) has no use case. 1.2 `Sign In` and 1.3 `Reset Password` are modelled; session termination is not. Minor, but it is a stated requirement with no representation.

Two further observations:

6. Scheduler should probably not be an actor. It is an internal timing mechanism rather than an external entity with a goal, and modelling it as an actor risks presenting the system's own scheduling as an external stimulus.

7. The diagram is not legible at this size. Thirty-eight use cases and nine actors in one frame produces long association lines crossing the whole figure. Whatever its correctness, a reader cannot trace an actor to its use cases without effort.

3. Verification of the critique against the requirements

Each point was checked against `REQUIREMENTS.md` before being accepted or rejected. This section is our own analysis, not the AI's.

#	Claim	Verdict	Evidence
1	<<extend>> on the wrong base	Accepted	8.3.8 reads "at any time before completion" — the modelled relationship is narrower than the requirement

2	Notification path incomplete for residents	Accepted	5.2.8 and 8.5.2 confirmed; neither 3.4 nor 6.8 includes 4.3
3	No crew notification	Accepted	8.2.4, 8.2.6, 8.3.11 confirmed; no corresponding use case exists
4	Resident map use case missing	Accepted	§9 requirements carry no role restriction; §2.3 restricts only the dashboard and work orders (2.3.3)
5	Log out unmodelled	Accepted	2.1.12 confirmed
6	Scheduler is not a valid actor	Rejected	See §5
7	Diagram illegible at size	Already mitigated	Three role-scoped views were produced for this reason (DengueGuard-Resident/Operations/Crew-and-System)

4. The insightful point (lab §3.5.2)

Selected quote:

"6.7 Raise Issue on Job is modelled as extending 6.6 Complete Job with Evidence, but requirement 8.3.8 says an issue may be raised 'at any time before completion.' ... As drawn, a crew member can only raise an issue while completing the job — which is precisely the case the requirement excludes, since a crew member who finds an obstruction has by definition not completed anything."

Why this is the most valuable point.

It is the only critique that found a defect in the *semantics* of a relationship rather than a missing element. A missing use case is visible to anyone comparing two lists; this required reading what <<extend>> actually asserts — that the extending behaviour is inserted at an extension point inside the base use case's flow — and then testing that assertion against the timing words in a specific requirement. The two artefacts were individually defensible: the requirement says "at any time before completion", and the diagram's extension condition "(obstruction found)" is a plausible-looking label. The contradiction only exists in the relationship between them.

The consequence is real rather than cosmetic. The extend relationship as drawn would carry into the Lab 2 dialog map as a transition available only from the completion screen, and into Lab 3 as a control-class method reachable only from the completion path — so a crew member arriving at a locked construction site would have no way to report it without first claiming to have completed work they have not done. That is a genuine usability failure that would have surfaced in the Lab 5 demo, and it would have been expensive to unpick at that point because the state machine and the screens would both already be built on it.

Action taken. 6.7 Raise Issue on Job is being re-modelled as a direct association from Cleaning Crew Member, valid against any work order not yet Completed, and the change is logged for the Lab 2 refinement of the use case model.

5. The unhelpful point (lab §3.5.2)

Recorded as well as the insightful one, because it illustrates a failure mode worth knowing about.

Selected quote:

"Scheduler should probably not be an actor. It is an internal timing mechanism rather than an external entity with a goal..."

Why it does not hold.

This is a stylistic objection presented with the same confidence as the five genuine defects, and it is wrong on the modelling convention. Time and scheduled triggers are conventionally modelled as actors precisely when they initiate a use case that no human initiates — which is the situation here: requirements 1.1.1, 1.2.1, 1.3.1 and 4.1.2 all specify recurring system behaviour on a fixed interval with no user in the loop. Removing the Scheduler actor would leave 7.1 through 7.7 with no initiator at all, which is a worse model, not a cleaner one. The Scheduler is also load-bearing for this project specifically: the "live data update" criterion the module grades is exactly what those seven use cases represent, and hiding their trigger would obscure the one thing the model most needs to show.

The general lesson. The AI gave no signal distinguishing its five verified defects from its one unfounded stylistic preference — the same declarative register, the same structure, no hedging on either. An AI critique is a list of *candidate* defects, and each has to be checked against the source before it is acted on. Point 6 would have degraded the model if applied on the AI's authority; points 1 to 5 improved it. Nothing in the response's tone separated them.

6. Changes made to the use case model as a result

Change	Origin
6.7 Raise Issue on Job re-modelled as a direct association, not an extension of 6.6	Point 1
3.4 Moderate Report and 6.8 Verify Completed Work to <<include>> 4.3 Notify Resident	Point 2
New use case 4.4 Notify Crew Member, included by 6.3 Assign Work Order and 6.8 Verify Completed Work	Point 3

New use case 2.4 View Cluster Map associated with Resident	Point 4
New use case 1.5 Sign Out associated with Registered User	Point 5
No change to the Scheduler actor	Point 6, rejected
No further change for legibility — role-scoped views already exist	Point 7

Scheduled for the Lab 2 refinement of the use case model, together with the 18 use case descriptions already inventoried but not yet written.

7. Declaration of AI use

Claude (Opus 5) was used to produce this critique, as directed by lab procedure §3.5. Every claim in §2 was independently checked against `REQUIREMENTS.md` before acceptance, and one of the seven was rejected on verification (§5). The requirements, use case model and data dictionary were drafted by the team with AI assistance and are understood and defensible by the team members who submitted them.